

AN INTRODUCTION

Object-Oriented Analysis & Design

An introduction to the discipline of modelling
software *before* you build it.

Sunder Ali Khowaja

School of Computing ·

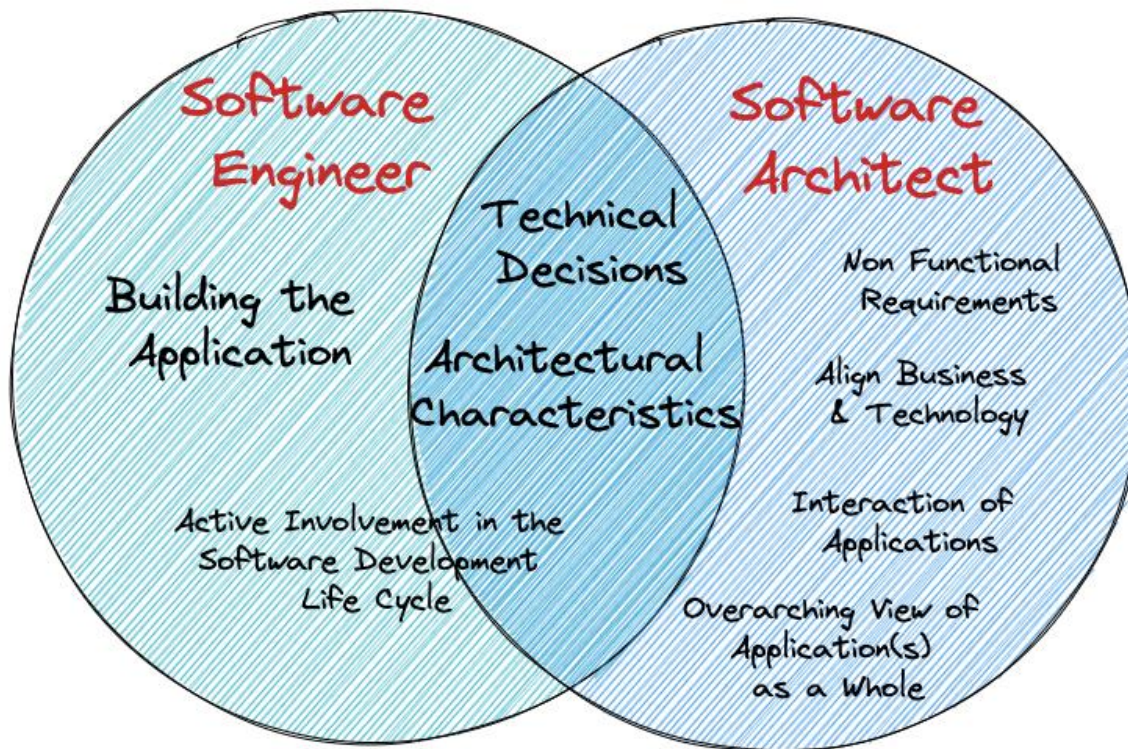
Semester 1



DUBLIN CITY UNIVERSITY

Software Architect

Typical "Breakdown of Responsibilities" Between the Software Engineer and Software Architect Across the Industry



(c) CloudWayDigital.com



Software Designer

Describe a car to someone who has *never* seen

Try it. What words do you reach for first?

You'd probably describe its parts like wheels, doors, and engine. You'd describe what those parts do like wheels roll, doors open, the engine turns fuel into motion. And you'd describe how those parts fit together; the engine connects to the wheels through a gearbox.

"That act of describing, choosing what to say, what to leave out, what level of detail to give, is exactly what object-oriented analysis and design is."

→ *Hold that thought. We'll come back to it.*

- **MODULE GOALS**

What This Module Is Trying to Achieve

1 Core Ideas

Introduce the main ideas in Object-Oriented Analysis & Design.

2 Practice, Agile-Style

Apply those ideas hands-on in a group project, using an Agile approach (Scrum).

3 UML as a Thinking Tool

Use UML diagrams to think through and describe designs, not just draw pictures after the fact.

SCOPE NOTE

We won't build a fully polished, production-ready system, the focus is prototyping. But by the end, your game should run!

You've already used a dozen pieces of software today

Alarm · music on the way here · mobile payment · chat · the campus turnstile · this slide deck... software quietly runs transport, hospitals, banks, games, and the phone in your pocket.

 “Think back to this morning, how many pieces of software did you interact with before reaching this room?”

We only notice software when it breaks. And when it breaks at scale, the story is rarely “a bad programmer”, it's usually something that happened much earlier...

Why does software need analysis & design?

Because the most expensive mistakes in software are made long before anyone writes a line of code.



What do these have in common?



- *Ariane 5 Flight 501 lifts off, 4 1996. Thirty-seven seconds was debris.*

\$370M

lost

CASE 01 Ariane 5 Flight 501 · 4 June 1996

Reuse without re-analysis

Guidance code from Ariane 4. A 64-bit floating-point velocity value was converted to a 16-bit signed integer, overflowed, and threw an unhandled exception. The rocket veered off course and was destroyed by its self-destruct system.

\$125M

lost

CASE 02 Mars Climate Orbiter · 1998

Two teams, two units

Two engineering teams used different units. One used pound-force-seconds for impulse, the other used newton-seconds. The mismatch was discovered only after the orbiter burned up in Mars' atmosphere.

6+

**Patients
Killed**

CASE 03 Therac-25 medical accelerator · 1985–87

Software trusted over hardware

A software race condition let the machine fire its high-power beam in 'unsafe mode'. Hardware interlocks had been removed, in the belief that software was reliable enough to replace them.

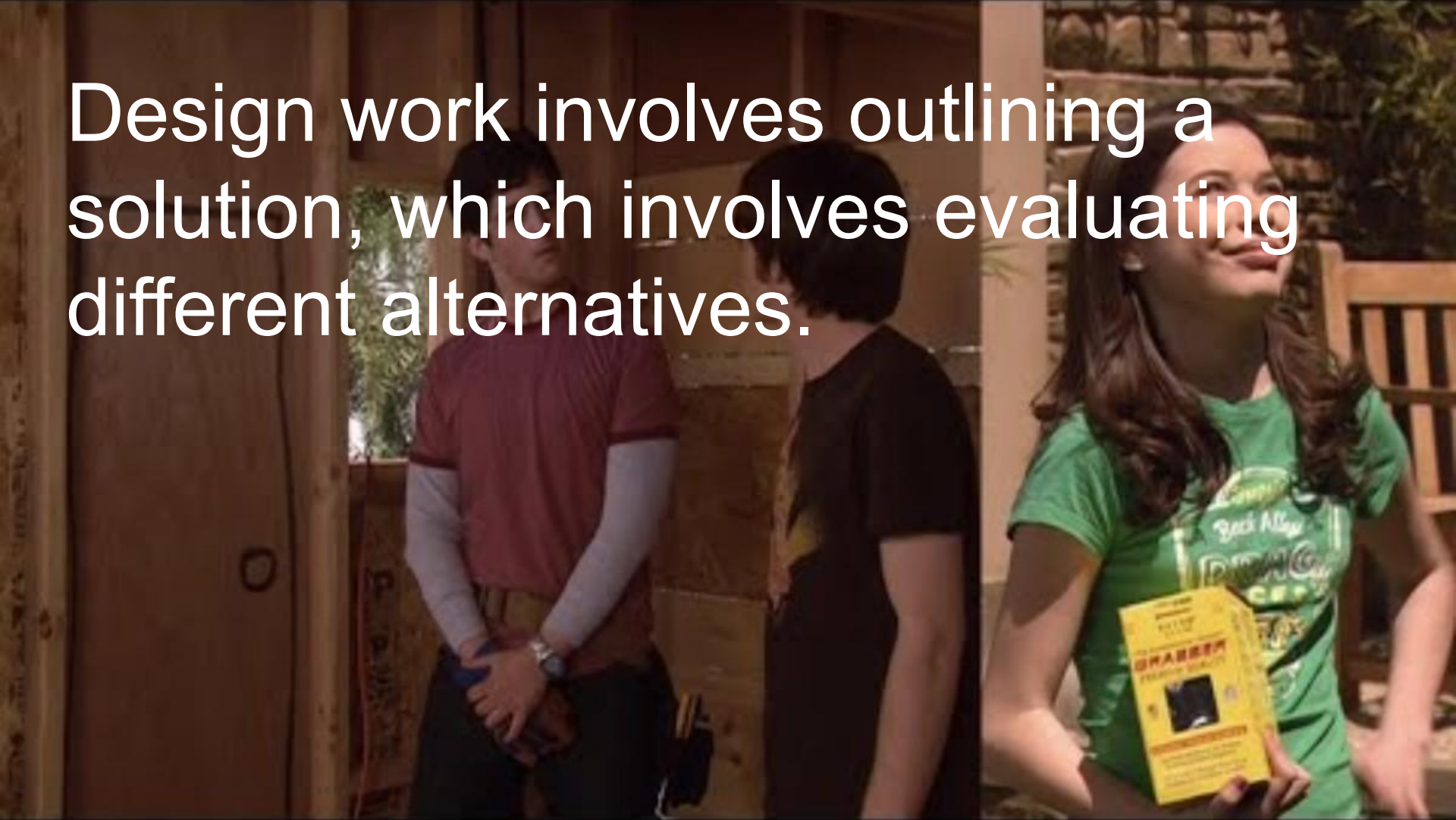
\$460M

lost

CASE 04 Knight Capital Group – August 2012

A botched software deployment sent unintended orders to the market. In under an hour, the firm lost roughly \$460M, nearly bankrupting it.

Design work involves outlining a solution, which involves evaluating different alternatives.



? Live poll (hands up): why do software projects fail?

A Programmers made too many coding mistakes

B Requirements & communication problems

C The wrong programming language or technology

D Bad luck

THE EVIDENCE: The Standish Group “CHAOS” studies (tracking IT projects since 1994):

- 2020 results: only 31% of projects succeeded; 50% were “challenged” (late, over budget or reduced scope); 19% failed outright.
- Top success factors across the studies: user involvement, executive management support, and a clear statement of requirements.
- Top failure factors mirror these: lack of user input, incomplete requirements, changing requirements.

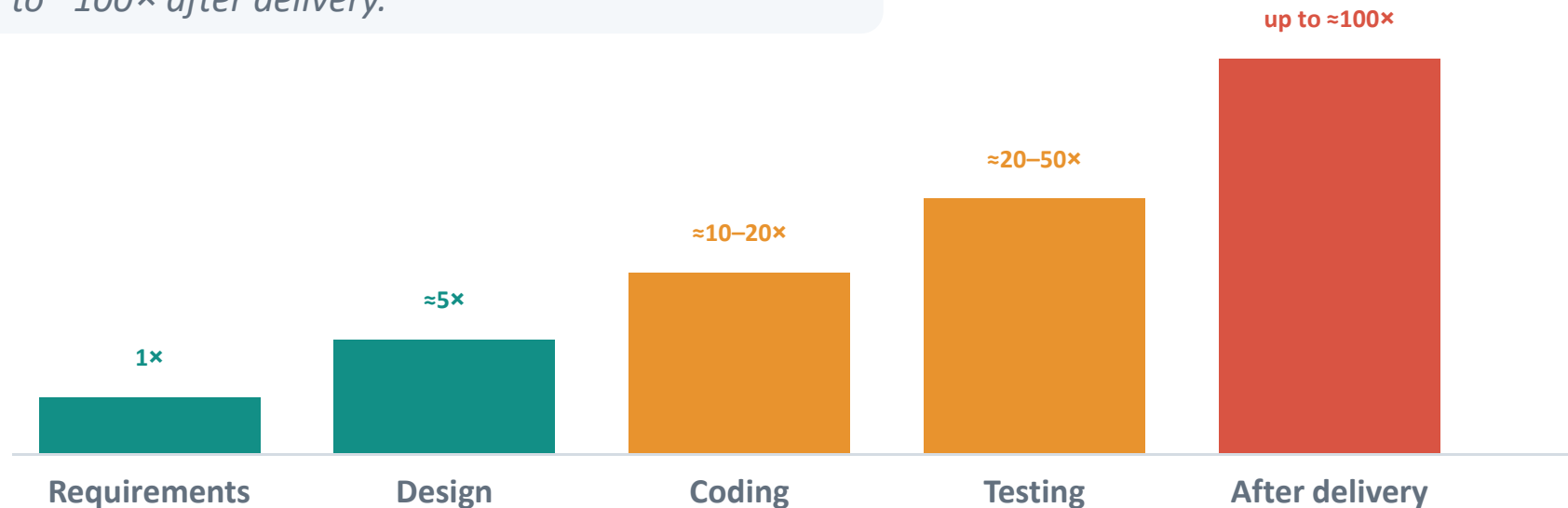
Notice: none of the top success factors is “typed faster”. They are analysis & design skills: and that's what this module teaches.

And mistakes get more expensive the later we catch them

The classic “cost of change” curve

Relative cost to fix a defect that was introduced during requirements.

After Boehm (1981) and Boehm & Basili (2001): up to $\sim 100\times$ after delivery.



This module lives at the left-hand end of that graph: think early, when changes are still cheap.

Object-oriented thinking

Modelling the world as collaborating objects with state, behaviour and identity, before we write any code.



Why Think in Objects?

PROCEDURAL THINKING

- Data and the functions that use it live separately.
- A program is a sequence of steps to run, top to bottom.
- Logic tends to be hard to reuse across features.

OBJECT-ORIENTED THINKING

- Data and behaviour are bundled together into objects.
- A program is a set of objects collaborating with each other.
- Objects mirror real-world things and become reusable building blocks.

Commonly-cited benefits of the OO approach: modularity, easier maintenance, greater reuse, and models that map more naturally onto real-world entities.

The software *development cycle*.

Six recurring phases, every non-trivial software project moves through them, often repeatedly.

01

Planning & Requirements

Decide what to build and why. Talk to users, write the SRS (Software Requirements Specification).

02

Analysis

Structure the problem. Identify the key entities, the use cases, the constraints.

03

Design

Decide how the system will be organized, classes, components, interfaces, architecture.

04

Implementation

Write the code. Translate the design into a running program.

05

Testing & Integration

Verify the system behaves as intended. Find defects early, fix them, re-test.

06

Maintenance

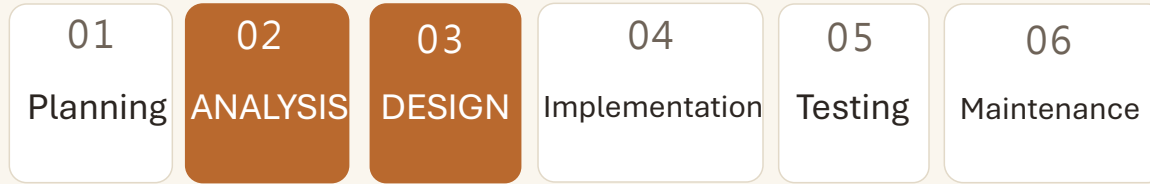
Keep the system useful after release, bug fixes, enhancements, retiring legacy pieces.



In practice these phases overlap, iterate, and repeat. Waterfall is the textbook version; Agile (which you'll the group project) is the iterative version.

Where OOA&D *lives* in the cycle.

} THE SIX PHASES · 02 & 03 ARE OURS



02 · Analysis WHAT

What must the system do, for whom, under what constraints.

03 · Design HOW

How is it organized, what objects exist, how do they collaborate.



The two phases this course is really about. Analysis answers **WHAT**: what does the system need to do, for whom, under what constraints. Design answers **HOW**: how is the system organised, what objects exist, how do they collaborate. Implementation, testing, and maintenance follow, but they inherit the quality of the analysis and design that came before them.

"A bad design can be coded perfectly. A good design can survive bad code."

Analysis ≠ Design ≠ Implementation

The house metaphor: three different conversations

	Analysis	Design	Implementation
The question	WHAT must the system do?	HOW should it do it, conceptually?	HOW do we build it in code?
Focus	The problem space: needs, domain, requirements	The solution space: structure & behaviour of the system	Languages, frameworks, databases
Typical output	Requirements & domain models (use cases, candidate classes)	Design models (class diagrams, interactions, state)	A working program
House metaphor	Agreeing what the client needs	The architect's blueprints	The construction crew

Memorise this: Analysis = problem space. Design = solution space. Mixing them up is how projects get confused.

What is "object-*oriented*"?

Five words you'll hear for the rest of your career: object, class, ADT, encapsulation, interface.

Model the world as *objects* that *collaborate*.

An object-oriented program is a **community of objects**. Each one knows (its state) and does something (its behaviour). They send messages to

Your job, as analyst and designer, is to decide four things:

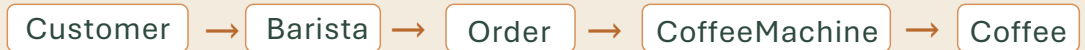
- 01 which objects should exist
- 02 what each one should know
- 03 what each one should do
- 04 how they should talk to each other

REAL-WORLD ANALOGY



A coffee shop, modelled as objects.

A Customer wants coffee. They don't reach into the themselves; they ask the Barista. The Barista takes the Order, to the CoffeeMachine, returns the Coffee to the Customer. In terms: Customer, Barista, Order, CoffeeMachine, Coffee, five each with its own state and behaviour, exchanging messages.



5 objects · each with state & behaviour · exchanging messages

→ *Same idea scales from a coffee shop to Amazon. The vocabulary is the*

Objects are everywhere, literally

The core OO idea starts with looking at the world differently

Look around this room.

The projector. The whiteboard. Your laptop. You.

In OO thinking, each of these is an object:

- State: the data it holds right now
- Behaviour: the things it can do
- Identity: this one, not that one

Example object: this whiteboard

State:

text written on it, marker colours available, isClean

Behaviour:

write(message), erase(), changeMarker(colour)

Identity:

this whiteboard: not the one in the room next door

? Your turn: name two more objects in this room: with one piece of state and one behaviour each.

The basic principle of object orientation: build programs from sets of collaborating, interacting objects, each modelling something real.

A class is the blueprint for objects.

One class, many objects. The class defines; the objects instantiate.

A class is a template. It defines what data its objects will hold and what operations they will perform. You don't drive the class `Car`; you drive an instance of `Car`. The class is the recipe; the object is the cake.

A class implements the spec of what its objects do.

Abstract Data Type (ADT)

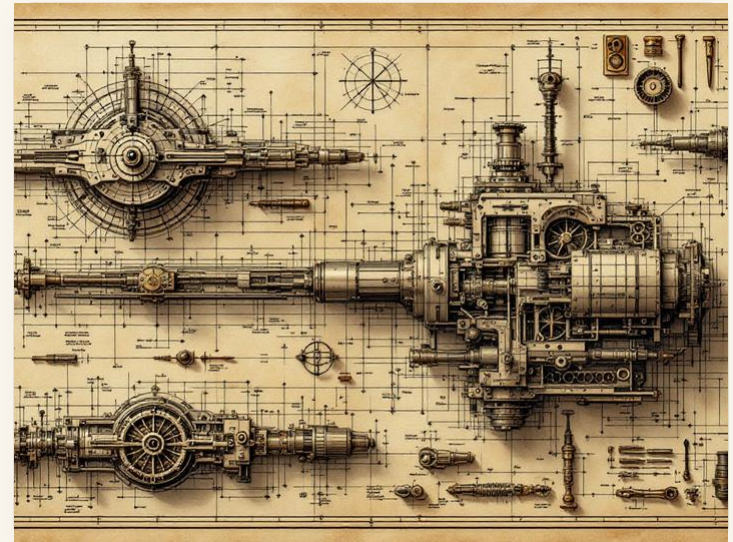
A class provides the of that behaviour, i.e., the actual code.

Implementation

A class is used to create objects

Instances

An Abstract Data Type (ADT) specifies WHAT each operation does, but not HOW. A class is the ADT plus its implementation.



A blueprint specifies what to build, not how every screw is machined. A class does the same for its objects.

Four words you'll hear a lot this semester

A preview, we'll unpack each one properly in later lectures

Abstraction

Capture the essentials, deliberately ignore the rest.

 *A Luas map: not to scale, exactly as detailed as it needs to be.*

Encapsulation

Hide the internals; offer a clean, stable interface.

 *Your phone: buttons and screen, not soldering irons.*

Inheritance /generalisation

Specialise what already exists.

 *A Student is a Person, with a few extra responsibilities.*

Polymorphism

Same request, different behaviour per type.

 *One play() command works for songs, videos and playlists.*

🕒 DESIGN CHALLENGE: 2 minutes · think → pair → share

Pick any app you use every day: music, maps, food delivery, a game.

1. Name three objects/classes it probably contains.

2. For each: one piece of state (data it remembers).

3. For each: one behaviour (something it can do).

4. Describe one interaction between two of them.

Example start:

Food-delivery app:
Order, Restaurant,
Courier...

*Order.state = items, total,
status*

Order.track()

*Courier receives Order from
Restaurant*

RUNNING EXAMPLE

Meet our running example: Add to Wishlist.

A single e-commerce feature, rich enough to be interesting

CONTEXT An e-commerce company wants to let customers save items they are interested in but not ready to buy yet. The business goal is to boost engagement and repeat purchases.

THE NAIVE ASK

"Just let users save items they like."

THE ANALYST JOB

Turn that vague ask into precise, structured, agreed-upon requirements that the team can build to.

Three deliverables come out of analysis:

- Requirements specification (THE WHAT)
- Primary class candidates (THE NOUNS)
- Use case diagrams (THE VERBS)

TRACE · STEPS 1 AND 2

From a vague ask to a structured plan.

Same feature. Two perspectives. Two outputs.

01 · ANALYSIS



Decide **what** the system must do.

- What information does a wishlist store? Product name, price, product image, date added.
- What can the user do? Add item, remove item, share wishlist, view it later on any device.
- Who can see it? Only the owner, unless explicitly shared.
- Constraints? Wishlist must sync across devices; items that go out of stock should be flagged.

OUTPUTS

requirements specification · use case diagrams
· primary class candidates

02 · DESIGN



Decide **how** the system will be organised.

- Candidate classes: Wishlist, WishItem, User, ProductService.
- Relationships: a User has one Wishlist; a Wishlist contains many WishItems; WishItem references a Product via ProductService.
- UI: a heart icon on every product card; clicking toggles add/remove. A dedicated wishlist page accessible from the header.
- Storage: wishlist persisted per user in the existing user-data store; cached client-side for offline add.

OUTPUTS

class diagrams · object diagrams · sequence diagrams · UI mock-ups

TRACE · STEPS 3 AND 4

From plan to running code and then prove it works.

The design is now committed. The next two phases test whether it actually holds up.

03 · IMPLEMENTATION



Translate the design into running code.

- Build the API endpoints: POST /wishlist/items, DELETE /wishlist/items/{id}, GET /wishlist.
- Create the database tables: wishlists, wish_items, with foreign keys to users and products.
- Implement the UI: heart icon component, wishlist page, optimistic updates.
- Integrate with existing cart and checkout, adding from wishlist should pre-fill the cart.

OUTPUT

working code, deployable to a staging environment.

04 · TESTING & INTEGRATION



Verify the feature behaves as intended.

- Functional: every user action works as designed; edge cases (empty wishlist, deleted products).
- Performance: handles expected peak load (e.g., 10k concurrent users during a sale).
- Security: users cannot read or modify each other's wishlists; CSRF and XSS checks pass.
- Integration: wishlist plays nicely with cart, checkout, and product-catalog systems.

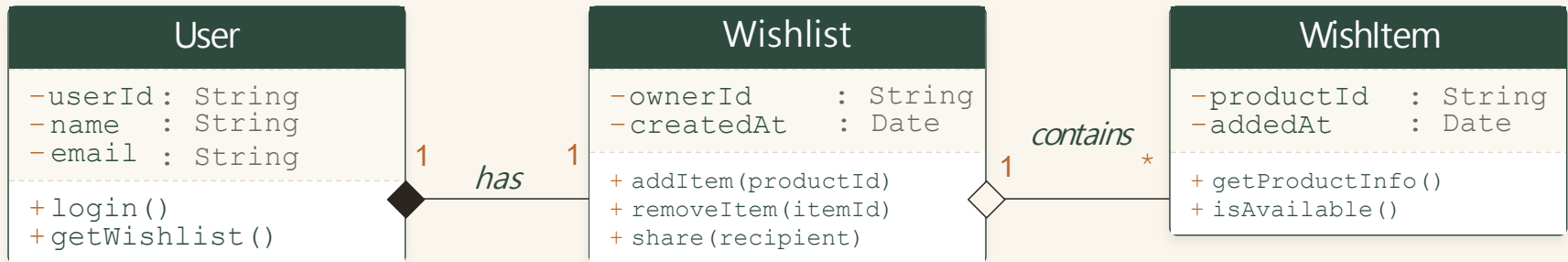
OUTPUT

a feature that's safe to release to real customers.

After release: maintenance begins. Customers will want new features (sharing, importing, exporting) and you will find minor bugs. Software is never done.

The same feature, drawn.

A UML class diagram of our wishlist feature. Don't worry about notation yet, just notice what becomes discussable.



COMPOSITION: filled diamond on the owner side

AGGREGATION: open diamond on the container side

Don't memorise the notation. Notice: the diagram makes the design discussable.

"Should Wishlist know about ProductService?" is now a precise question.

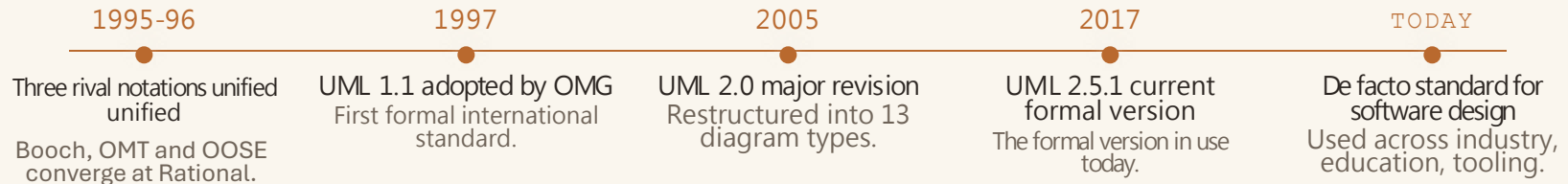
The language we'll speak: *UML.*

A shared visual vocabulary for software design: older than most of you.

HISTORY · CONTEXT

A brief history of UML.

Three rival notations. One unified language. Twenty-eight years and counting.



The Three Amigos

RATIONAL SOFTWARE · MID-1990s

Grady Booch

THE BOOCH METHOD

Visual and architectural emphasis.

James Rumbaugh

OMT: OBJECT MODELLING
TECHNIQUE

Data-centric static modelling.

Ivar Jacobson

OOSE: OBJECT-ORIENTED
SOFTWARE ENGINEERING

Use-case driven behavioural modelling.

Joined Rational Software in the mid-1990s. UML is the synthesis of their three methods. Now maintained by the OMG: Object Management Group, founded 1989.

■ Why model? And what exactly is UML?

Why model at all?

- You wouldn't build a house without blueprints, changes to a drawing are cheap; changes to concrete are not.
- Models communicate: to customers, teammates, and future-you.
- Models let you reason about behaviour before committing to code.
- Models are the documentation that keeps maintenance sane.

A model is an abstraction with a purpose: it captures what matters to a particular stakeholder: user, customer, designer...

UML: what it is / isn't

- ✓ A standardised visual modelling language, maintained by the Object Management Group (OMG).
- ✓ A family of diagram types covering structure and behaviour.
- ✓ Process-independent: pairs happily with Scrum, RUP, or your own workflow.

✗ Not a programming language.

✗ Not a method, it says what you can draw, never when or why.

✗ Not a substitute for thinking. Bad models model badly.

“Is UML a programming language?” (No. You can't run it: it's a language for thinking and communicating.)

CORE CONCEPT

One system. Many views.

Different stakeholders ask different questions. UML gives each a model that answers theirs.

A model is an abstraction of a system with a particular purpose: for example, to describe the behavioural aspects of the system to a particular category of stakeholder. A model doesn't try to capture everything; it captures what matters for one audience and one question.

Different models, supported by UML, of the same system are suitable for different viewpoints: the user, the customer, the requirements team, the designers, the deployers.

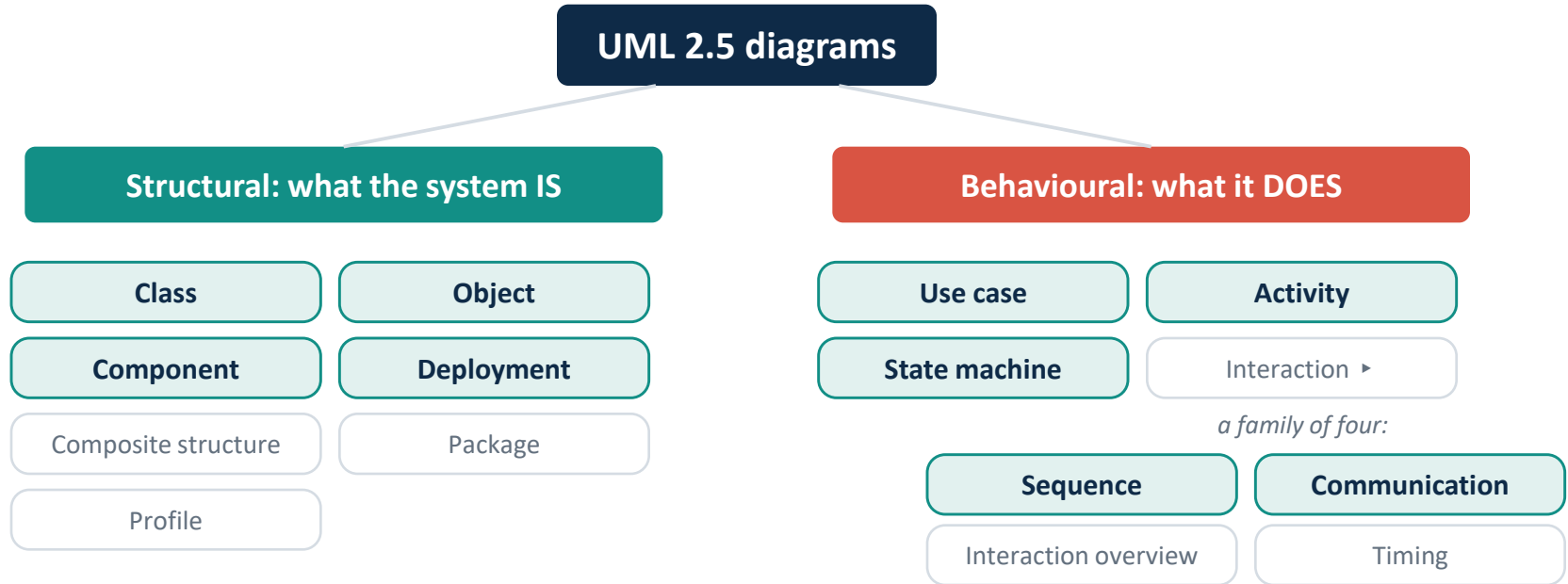
“A model captures a view of a physical system. Hence it is an abstraction, with a purpose.”

STAKEHOLDERS · THEIR QUESTION

😊	Customer	What does the system do for me?
😊	Requirements Engineer	What must the system do, precisely?
🔧	Designer	How is the system organised internally?
💎	Tester	What behaviours must I verify?
🏠	Ops Engineer	How is the system deployed and run?

■ The UML diagram family (v2.5)

Many diagram types but don't panic about the list



Solid chips = used in this module. You need the core; the rest are there when a special job calls for them.

Development Phase	Deliverable	Due Date	
Analysis (to week 4) (see §3.1.1,§3.1.2)	1. Refined requirements specification	UPLOAD: Fri. Oct. 2 nd at 17.00 (End of week 4)	
	2. Scenarios		
	3. Primary class list		
	4. Class diagrams		
	5. Use case diagrams		
	6. Result of “Structured walk-through”		
	7. Minutes/notes of team meetings		
Product Design (week 6) (see §3.2.1,§3.2.2)	1. Object diagrams	Should aim to have these parts done by Fri. Oct 16 th (end of week 6)	,
	2. Refined class diagrams		
	3. User interface mock-ups		
	4. State machines		
	5. Client server initial test		
Class Design (mainly weeks 5, 6– maybe some refinement in week 7) (see §3.2.3)	1. Communication diagrams	UPLOAD: Fri. Oct. 23 rd at 17.00 (End of week 7)	
	2. Sequence diagrams		
	3. Object diagrams		
	4. Refined class diagrams		
	5. Class skeletons		
	6. Minutes/notes of any team meetings		
Implementation (provisional) (weeks 8, 10) (see §3.3)	1. Source code (provisional)	UPLOAD Fri. Nov 13 th at 17.00 (End of week 10)	
	2. Minutes or notes of any team meetings		
Presentations (week 11-12) (see §3.4, §3.5)	1. Project presentation	UPLOAD: W/B Week 11 and 12, will be organised for in class	
Implementation (final) (week 12)	1. Summary of changes and additions since provisional source code delivery. 2. Minutes or notes of any team meetings	Fri. Nov. 27 th , at 17.00 (End of week 12)	

31

The group project: our semester-long case study



One substantial project assignment, chosen from two alternatives, detailed briefs to follow.



Teams of 6 students. Departures from that size only for strong, valid reasons.



Run it as an Agile (Scrum) team: plan, deliver, review, your meeting minutes and walk-throughs are part of the evidence.



All modelling in UML, produced with a diagramming tool.



Intermediate deliveries at fixed milestones.

 **ACTION: before Tuesday 15 September, 17:00:**

Any issue with the team, email the issue to sunderali.khowaja@dcu.ie

Group Project

The continual assessment element of CSC1046 will be based on a single project assignment (***from two alternatives***) to be carried out throughout the semester, with specified intermediate deliveries. The assigned project is a reasonably substantial one and is required to be done in teams.

Normally, teams will consist of **6 students**. Departures from this team size will only be allowed for strong, valid reasons. In case of any conflict of interest, the students should notify to sunderali.khowaja@dcu.ie by 17.00 on Tuesday Sept 15th

How you'll be assessed

Continuous assessment 25% of the module, all team-based

Analysis

5%

Product & Class Design

7%

Implementation (provisional + final)

8%

Presentation

5%

Good to know:

- Marks follow the milestone deliveries on the next slides.
- UML is expected to be produced with a diagramming tool.
- Meeting minutes count as evidence.

Total continuous assessment: 25% · each element delivered as a team

Full assessment rules are in the module handbook read them once, properly.

Reusing wisdom: design patterns & frameworks

You don't have to invent every solution from scratch

Design pattern

A template solution that developers have refined over time to solve a range of recurring problems.

Each pattern has: a name · a problem description · a solution · consequences.

Names you may overhear: Abstract Factory, Adapter, Bridge, Observer...

Framework

A set of classes providing a general solution that can be refined to provide an application or a subsystem.

Think: a furnished shell you customise — you decide the rooms, the structure is already standing.

Your project brief will ask you to consider frameworks: existing, modified, or new.

 1994: Gamma, Helm, Johnson & Vlissides, the “Gang of Four”, published 23 classic patterns, and pattern-thinking became a core OO skill.

Recipe vs shell: a pattern tells you how; a framework gives you where.

1.3.1 web portal for UML

- <http://www.uml.org/>
- Based on Object Management Group Standards Development Organization (OMG SDO)

A huge and ongoing project now at version 2.5.1.
(Now almost 28 years since 1.1)

Some References

Note: Some of the associated web-sites have examples of source code etc.

- 1) “Using UML - Software Engineering with Objects & Components”, Stevens & Pooley (Addison-Wesley)
- 2) “Object-Oriented Software Engineering, using UML, Patterns, and Java”, Bruegge & Dutoit (Pearson Prentice-Hall) [<https://uim.fei.stuba.sk/wp-content/uploads/2018/02/Object-oriented-Software-Engineering-3rd-Edition.pdf>]
- 3) “Software Design, from Programming to Architecture”, Braude (Wiley)
- 4) “Component-Based Software Engineering, putting the pieces together”, Heineman & Councill (Eds) (Addison-Wesley) [Mainly a background reference]
- 5) “Project-based Software Engineering, an object-oriented approach”, Stiller & LeBlanc (Addison-Wesley)
- 6) “Requirements Engineering, from system goals to UML models to software specifications”, van Lamsweerde (Wiley)
- 7) “Process-Centered Review of Object Oriented Software Development Methodologies”, R. Ramsin & R. Paige, ACM Computing Surveys, Vol. 40, No. 1, Article 3, February 2008
[https://sharif.edu/~ramsin/index_files/Publications_PDF/Ramsin_Paige_ACM_Computing_Surveys_2008_Process_Centered_Review.pdf]
- 8) “Data Structures and Algorithms in Java”, Goodrich & Tamassia (Wiley)